

COMP201: Software Engineering I

Object Oriented Design

Coursework Assignment 2 (2025/2026)

Software Design and Implementation

Assessment Information

Assignment number	2 of 2
Weighting	20%
Assignment Circulated date provided to class	9/11/2025
Deadline Day & Date & Time	4 th of December 2025 at 17:00
Submission Mode	Electronic
Learning outcome assessed	be fully aware of the principles and practice of an O-O approach to the design and development of computer systems; be able to apply these principles in practice. Be able to demonstrate how to effectively implement an O-O design in an O-O language such as Java or Python;
Submission necessary in order to satisfy Module requirements	No
Purpose of assessment	To assess the students ability to produce an OO design in UML and implement design
Marking criteria	See end of document
Late Submission Penalty	Standard UoL Policy

REPORT

Title page: put your name, your student number and the course on the first page.

TASK 1. (25%) Given the following informal specification, **identify good candidates** for classes and attributes, and **identify things that are outside** of the problem domain. You should use the noun identification technique and show your working. Also identify **all potential inheritance relationships**. You should ensure that data is **NOT** duplicated across classes even if a user places multiple bookings. Use the noun identification method of class elicitation for the first pass. For full marks please try and avoid as much as possible duplication of data within any class.

Present your design as a class diagram, including all relevant attributes and relationships

Your customer is a travel agency that wants a reservation system that will run on the Internet. This reservation system will allow clients to keep track of all their travel reservations for airlines, hotel, rental cars and travel insurance. The client must enter the names of all his/her traveling companions, but all reservations will be referenced by the primary client. The system needs to make it easy for a client to have multiple reservations. All reservations will include a booking number as well as a reference to the names, passport numbers and dates of birth of all the travelers involved in the reservation. The system should also have an address for the primary client. Airline reservations will include the airline, flight number, class of seat and travel date and time. Hotel reservations will include the type (twin, single, double) of room, the start date and the end date as well as the name and address of the hotel. Car rental reservations will include the type of car requested, start date, days of hire and the drivers' license numbers. The insurance booking will have a start date, an end date and level of cover bronze, silver or gold.

TASK 2. (25%)

Produce an implementation of the solution in the Java programming language. For your implementation make sure your classes have weak coupling and are immutable. For your solution you do NOT need to add complex algorithms but just implement the data design and provide some basic validation for the object construction. For your Java files zip then up into a file called source.zip and upload.

TASK 3. (25%) You are required to draw a UML **activity diagram** to represent the following scenario of a hairdresser's salon.

Customers enter the salon and wait until the next hairdresser is free. They then indicate whether they would like their hair washed first or a "dry-cut" without having their hair washed. The hairdresser washes the hair (if asked for) and then cuts it. After finishing the customer's hair the hairdresser moves onto the next waiting customer, or waits for another one to enter the salon. The customer goes to the till and waits for a cashier to be free to take their payment.

They can pay by either cash or by credit card (where they need to type their pin into the machine) and they then leave the salon.

TASK4. (25%) Draw a UML **sequence diagram** that specifies the following protocol of initiating a two-party phone call. NOTE: ArgoUML does not fully support Sequence Diagrams, it may be better to use a different program (such as OpenOffice Draw/ Microsoft Powerpoint) or (neatly) draw the diagram by hand. Let us assume that there are four objects involved:

- two Callers (s and r),
- an unnamed telephone Switch, and
- Conversation (c) between the two parties.

The sequence begins with one Caller (s) sending a message (liftReceiver) to the Switch object. In turn, the Switch calls setDialTone on the Caller, and the Caller iterates (7 times) on the message dialDigit to itself. The dialled digits are then sent to the Switch. The Switch object then calls itself with the message routeCall. It then creates a Conversation object (c), to which it delegates the rest of the work. The Conversation object (c) rings the Caller (r), who asynchronously sends the message liftReceiver. The Conversation object then tells both Caller objects to connect, after which they talk. Once Caller (r) sends a disconnect message to Conversation then Conversation tells both Caller objects to disconnect and also it tells the Switch to disconnect. After that Switch deletes the object Conversation.

All the key words you need to include are underlined – do *not* invent any details additional to those given above.

Marking Criteria

Task	A++ to A 70%+	B 60%-69%	C 50%-59%	D 40%-49%	E+ 35%-39%	E- to G < 35%
1	Well chosen classes for the scenario and a description of any removed superfluous classes. Correct inheritance used.	Mostly correct classes chosen but without solid justification. Inheritance relations denoted correctly.	Inappropriate classes or inheritance relations defined with minor omissions and/or poor justification of chosen classes.	Major omissions of classes and incorrectly chosen/missing inheritance relations.	Some understanding of classes and how they should be derived but with a poor choice of classes and no justification	No evidence of understanding the concept of deriving classes from a scenario.
2	Excellent well written, styled and commented code with excellent validation.	Well written code but with some comments missing and some issues with validation.	Some classes missing.	Poorly structured code.	Many mistakes lots of classes missing.	No serious attempt to program the code.
3	Correct notation and a good level of abstraction used throughout	Mostly correct notation but with minor errors and/or minor omissions for the modelling of the scenario	A good attempt to model the scenario but with slightly incorrect notation and an inadequate level of detail.	Insufficient level of detail but some evidence of correct understanding of activity diagrams.	Some evidence of understanding activity diagrams and an attempt to model the scenario in some meaningful way.	No serious attempt to model the scenario with an activity diagram.
4	Correct notation and a good level of abstraction used throughout	Mostly correct notation but with minor errors and/or minor omissions for the modelling of the scenario	A good attempt to model the scenario but with slightly incorrect notation and an inadequate level of detail.	Insufficient level of detail but some evidence of correct understanding of sequence diagrams.	Limited evidence of understanding sequence diagrams and their uses.	An inadequate attempt to use sequence diagrams in a meaningful way.